

Tài liệu tự học DevOps & CI/CD cho người mới bắt đầu

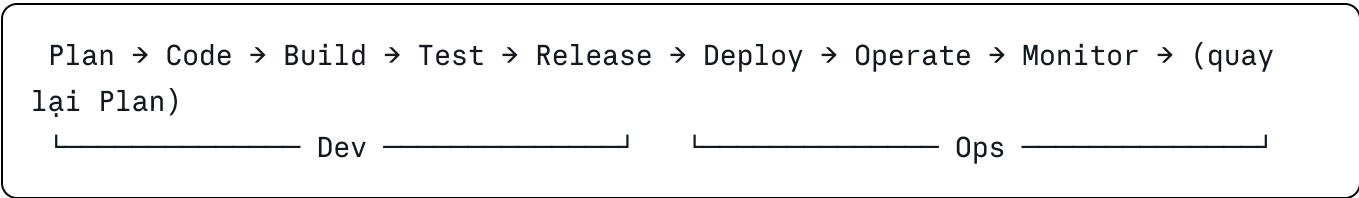
Nội dung: DevOps · CI/CD · GitHub Actions · Workflow · Docker **Demo xuyên suốt:** Webapp Python (FastAPI) + pytest + Docker + GitHub Actions (build → test → deploy)

PHẦN I — TÓM TẮT LÝ THUYẾT

1. DevOps là gì?

DevOps = Development (Dev) + Operations (Ops) — là văn hóa và tập hợp thực hành nhằm rút ngắn chu kỳ phát triển phần mềm, đưa sản phẩm đến người dùng **nhANH hơn, thường xuyên hơn và tin cậy hơn**, bằng cách xóa bỏ rào cản giữa đội phát triển (viết code) và đội vận hành (triển khai, giám sát hệ thống).

Vòng đời DevOps (DevOps lifecycle) thường được vẽ thành vòng lặp vô tận (infinity loop):



Các nguyên tắc cốt lõi:

Nguyên tắc	Ý nghĩa
Automation	Tự động hóa mọi thứ lặp lại: build, test, deploy
CI/CD	Tích hợp và chuyển giao liên tục (trọng tâm tài liệu này)
Infrastructure as Code (IaC)	Hạ tầng định nghĩa bằng code (Dockerfile, Terraform...)
Monitoring & Feedback	Giám sát liên tục, phản hồi nhanh
Collaboration	Dev và Ops chia sẻ trách nhiệm chung

Công cụ tiêu biểu theo từng giai đoạn: Git/GitHub (code) → GitHub Actions, Jenkins, GitLab CI (build/test/deploy) → Docker, Kubernetes (đóng gói, chạy) → Prometheus, Grafana (giám sát).

2. CI/CD là gì?

2.1 CI — Continuous Integration (Tích hợp liên tục)

Lập trình viên **thường xuyên merge code vào nhánh chính** (nhiều lần mỗi ngày). Mỗi lần merge, hệ thống **tự động build và chạy test**. Mục tiêu: phát hiện lỗi sớm, khi lỗi còn nhỏ và dễ sửa.

Quy trình CI diễn hình khi bạn `git push`:

- 1. Server CI phát hiện thay đổi (event)
- 2. Lấy code mới nhất (checkout)
- 3. Cài dependencies, build
- 4. Chạy test tự động (unit test, lint)
- 5. Báo kết quả  /  (trên PR, email, Slack...)

2.2 CD — Continuous Delivery vs Continuous Deployment

Hai khái niệm dễ nhầm, khác nhau ở **bước cuối cùng**:

	Continuous Delivery	Continuous Deployment
Build, test tự động		
Đóng gói artifact (Docker image) tự động		
Deploy lên production	Thủ công — người bấm nút duyệt	Tự động hoàn toàn — pass test là lên production

2.3 Pipeline CI/CD

Pipeline = chuỗi các bước tự động nối tiếp nhau. Pipeline mà ta sẽ xây trong phần Lab:

```
git push → [CI: Lint + Test] → [Build Docker image] → [Push image lên Registry] → [Deploy]
                (fail thì dừng, báo lỗi ngay tại đây)
```

Lợi ích: phát hiện bug sớm · release nhanh và đều đặn · giảm rủi ro deploy (deploy nhỏ, thường xuyên) · quy trình lặp lại được, không phụ thuộc "người biết deploy".

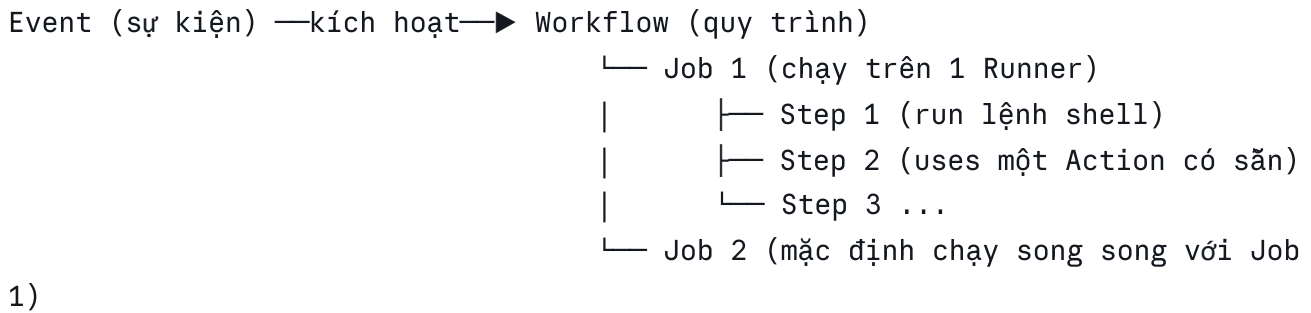
3. GitHub Actions

3.1 GitHub Actions là gì?

GitHub Actions là nền tảng CI/CD tích hợp sẵn trong GitHub, cho phép tự động hóa build, test, deploy ngay trong repository — không cần cài server CI riêng. Miễn phí 2.000 phút/tháng cho repo private và **không giới hạn cho repo public**.

 Tài liệu chính thức: <https://docs.github.com/en/actions>

3.2 Các khái niệm cốt lõi



Khái niệm	Định nghĩa
Workflow	Quy trình tự động, định nghĩa bằng file YAML đặt trong thư mục <code>.github/workflows/</code> của repo. Một repo có thể có nhiều workflow.
Event	Sự kiện kích hoạt workflow: <code>push</code> , <code>pull_request</code> , <code>schedule</code> (theo lịch cron), <code>workflow_dispatch</code> (bấm nút chạy tay)...
Job	Nhóm các step, chạy trên một máy ảo mới tinh (runner). Các job mặc định chạy song song ; muốn tuần tự thì dùng <code>needs</code> .
Step	Một bước trong job: hoặc chạy lệnh shell (<code>run</code>), hoặc dùng lại action có sẵn (<code>uses</code>). Các step trong 1 job chạy tuần tự , chia sẻ cùng máy ảo.
Action	Đơn vị tái sử dụng — "thư viện" cho workflow. Ví dụ <code>actions/checkout</code> (tải code về runner), <code>actions/setup-python</code> (cài Python). Tìm trên GitHub Marketplace.
Runner	Máy chủ thực thi job. GitHub cung cấp sẵn (<code>ubuntu-latest</code> , <code>windows-latest</code> , <code>macos-latest</code>) hoặc tự host (self-hosted).

3.3 Giải phẫu một file workflow — giải thích TỪNG PHẦN

File workflow **phải** đặt trong `.github/workflows/`, đuôi `.yml` hoặc `.yaml`. Dưới đây là một workflow mẫu với chú thích từng dòng:

```
yaml
```

```

# ===== PHẦN 1: ĐỊNH DANH =====
name: CI Demo
# `name`: tên workflow, hiển thị trên tab "Actions" của repo.
# Nếu bỏ trống, GitHub hiển thị đường dẫn file thay thế.

# ===== PHẦN 2: TRIGGER (KHI NÀO CHẠY) =====
on:
    # `on`: khai báo (các) event kích hoạt workflow
    push:
        # chạy khi có commit được push...
        branches: [ main ]
        # ...nhưng chỉ trên nhánh main (lọc theo nhánh)
    pull_request:
        # chạy khi có PR nhắm vào main
        branches: [ main ]
    workflow_dispatch:
        # thêm nút "Run workflow" để chạy thủ công trên UI

# ===== PHẦN 3: BIÊN MÔI TRƯỜNG (tùy chọn) =====
env:
    # biên dùng chung cho TOÀN BỘ workflow
    APP_NAME: demo-app
    # truy cập trong step bằng $APP_NAME

# ===== PHẦN 4: JOBS (LÀM GÌ) =====
jobs:
    test:
        # ID của job (tự đặt, duy nhất trong workflow)
        name: Run tests
        # tên hiển thị trên UI (tùy chọn)
        runs-on: ubuntu-latest
        # runner: máy ảo Ubuntu mới nhất do GitHub cấp

    steps:
        # danh sách các bước, chạy TUẦN TỰ
        # --- Step dùng action có sẵn: từ khóa `uses` ---
        - name: Checkout source code
          uses: actions/checkout@v4
          # actions/checkout = action chính thức tải code repo về runner.
          # @v4 = phiên bản. LUÔN ghim version để tránh vỡ khi action đổi.
          # Không có step này, runner là máy trắng, KHÔNG có code của bạn!

        - name: Set up Python
          uses: actions/setup-python@v5
          with:
            # `with`: truyền tham số (input) cho action
            python-version: "3.12"

        # --- Step chạy lệnh shell: từ khóa `run` ---
        - name: Install dependencies
          run: pip install -r requirements.txt

        - name: Run tests
          run: pytest -v
          # test fail (exit code != 0) → step fail → job fail

```

```

deploy:
  runs-on: ubuntu-latest
  needs: test          # `needs`: chỉ chạy SAU KHI job `test` thành công
                      # (không có needs → 2 job chạy song song)
  if: github.ref == 'refs/heads/main' # điều kiện: chỉ deploy trên main
  steps:
    - name: Deploy
      run: echo "Deploying ${ env.APP_NAME }..."
      # ${ ... } là EXPRESSION: truy cập context như env, github, secrets

```

Tóm tắt các từ khóa quan trọng nhất:

Từ khóa	Vai trò
<code>name</code>	Tên workflow / job / step (hiển thị trên UI)
<code>on</code>	Event kích hoạt (push, pull_request, schedule, workflow_dispatch...)
<code>jobs</code>	Danh sách các job
<code>runs-on</code>	Loại runner (ubuntu-latest phổ biến nhất)
<code>steps</code>	Các bước trong job
<code>uses</code>	Dùng action có sẵn (<code>owner/repo@version</code>)
<code>with</code>	Truyền input cho action
<code>run</code>	Chạy lệnh shell trực tiếp
<code>needs</code>	Khai báo phụ thuộc giữa các job (tạo pipeline tuần tự)
<code>if</code>	Điều kiện chạy job/step
<code>env</code>	Biến môi trường (mức workflow / job / step)
<code>\${{ secrets.X }}</code>	Đọc secret — thông tin nhạy cảm (token, password) lưu trong Settings → Secrets của repo, không bao giờ lộ trong log

 Tham khảo đầy đủ cú pháp: <https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-syntax>

4. Docker

4.1 Docker là gì? Vì sao cần?

Vấn đề kinh điển: *"Code chạy trên máy em mà!"* — app chạy tốt trên máy dev nhưng lỗi trên server vì khác phiên bản Python, thiếu thư viện, khác OS...

Docker giải quyết bằng cách đóng gói ứng dụng **cùng toàn bộ môi trường chạy** (Python, thư viện, cấu hình) vào một **container** — chạy giống hệt nhau ở mọi nơi: máy dev, máy CI, server production.

4.2 Ba khái niệm nền tảng

Khái niệm	Định nghĩa	Ví von
Image	Bản mẫu chỉ đọc (read-only), chứa app + môi trường. Build một lần, chạy nhiều nơi.	Công thức nấu ăn / khuôn bánh
Container	Một instance đang chạy của image, cô lập với hệ thống. Một image tạo được nhiều container.	Món ăn nấu từ công thức
Registry	Kho lưu trữ image trên mạng: Docker Hub, GitHub Container Registry (GHCR)... <code>docker push</code> để đẩy lên, <code>docker pull</code> để kéo về.	GitHub nhưng cho image

Container vs Máy ảo (VM): container **chia sẻ kernel** với máy chủ, chỉ đóng gói app + thư viện → nhẹ (MB thay vì GB), khởi động trong giây thay vì phút.

4.3 Dockerfile — giải thích TỪNG THÀNH PHẦN

Dockerfile là file văn bản chứa các **chỉ thị (instruction)** hướng dẫn Docker build image, từng dòng tạo thành từng **layer** xếp chồng lên nhau. Các instruction quan trọng nhất:

```
dockerfile
```

```
# ---- FROM: BẮT BUỘC, luôn đứng đầu ----
FROM python:3.12-slim
# Chỉ định IMAGE NỀN (base image) để xây tiếp lên.
# python:3.12-slim = Debian tối giản đã cài sẵn Python 3.12.
# Chọn bản "-slim" để image nhẹ (~120MB thay vì ~1GB bản đầy đủ).

# ---- WORKDIR: đặt thư mục làm việc ----
WORKDIR /code
# Mọi lệnh sau đó (COPY, RUN, CMD) đều thực thi trong /code.
# Nếu thư mục chưa tồn tại, Docker tự tạo. Tương đương "cd /code".

# ---- COPY: chép file từ máy bạn vào image ----
COPY requirements.txt .
# Cú pháp: COPY <nguồn: máy bạn> <đích: trong image>
# MẸO QUAN TRỌNG: copy requirements.txt TRƯỚC, copy code SAU.
# Docker cache theo layer – nếu requirements.txt không đổi thì layer
# cài thư viện được TÁI SỬ DỤNG, build lại chỉ mất vài giây.

# ---- RUN: chạy lệnh TẠI THỜI ĐIỂM BUILD image ----
RUN pip install --no-cache-dir -r requirements.txt
# Kết quả của RUN được "đóng băng" vào image.
# --no-cache-dir: không lưu cache của pip → image nhỏ hơn.

COPY ./app ./app
# Giờ mới copy source code (thứ thay đổi thường xuyên nhất) vào image.

# ---- ENV: đặt biến môi trường trong container ----
ENV PYTHONUNBUFFERED=1
# In log Python ra ngay lập tức, không buffer – dễ debug container.

# ---- EXPOSE: khai báo (tài liệu hóa) cổng container lắng nghe ----
EXPOSE 8000
# LƯU Ý: EXPOSE chỉ mang tính khai báo/tài liệu. Muốn truy cập từ ngoài
# vẫn phải map cổng khi chạy: docker run -p 8000:8000

# ---- CMD: lệnh chạy KHI CONTAINER KHỞI ĐỘNG (không phải lúc build) ----
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
# Dùng dạng exec (mảng JSON) thay vì chuỗi – giúp app nhận tín hiệu
# tắt máy (SIGTERM) và shutdown "êm" (graceful shutdown).
# --host 0.0.0.0: lắng nghe mọi network interface. Nếu để mặc định
# 127.0.0.1, app chỉ nghe trong nội bộ container → bên ngoài KHÔNG gọi được!
# Mỗi Dockerfile chỉ có 1 CMD hiệu lực (dòng CMD cuối cùng).
```


Phân biệt hay nhầm:

	RUN	CMD
Chạy khi nào	Lúc build image	Lúc khởi động container
Số lượng	Nhiều dòng	Chỉ dòng cuối có hiệu lực
Dùng cho	Cài thư viện, tạo thư mục	Lệnh khởi động app

File `.dockerignore` (đặt cạnh Dockerfile): liệt kê những gì **không** copy vào image (giống `.gitignore`) — giúp image nhỏ, build nhanh, không lộ secret:

```
venv/
__pycache__/
.git/
.env
*.pyc
```

Các lệnh Docker cần thuộc:

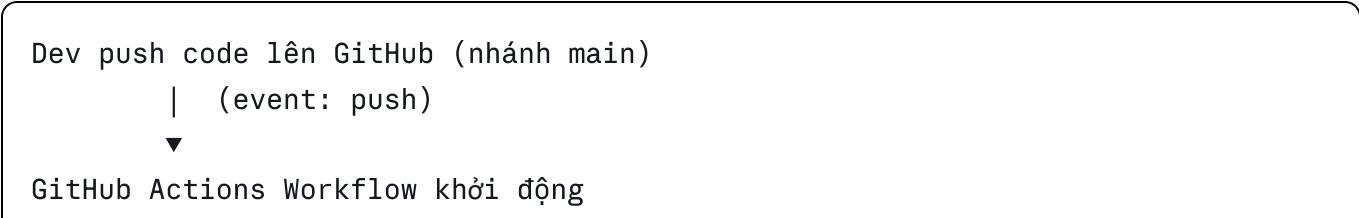
```
bash

docker build -t ten-image:tag .    # build image từ Dockerfile ở thư mục hiện tại
docker images                      # liệt kê image trên máy
docker run -d -p 8000:8000 ten-image  # chạy container (-d: chạy nền,
                                     # -p HOST:CONTAINER: map cổng)

docker ps                          # container đang chạy
docker logs <container-id>         # xem log
docker stop <container-id>         # dừng container
docker push / docker pull          # đẩy/kéo image với registry
```

 Docker docs: <https://docs.docker.com/get-started/> · Dockerfile reference: <https://docs.docker.com/reference/dockerfile/>

5. Ghép tất cả lại — Bức tranh CI/CD hoàn chỉnh



```
|
| └─ Job 1 – TEST (CI)
|   checkout code → cài Python → cài deps → chạy pytest + lint
|   ✖ fail → dừng pipeline, báo đỏ trên commit
|
| └─ Job 2 – BUILD & PUSH (needs: test)
|   build Docker image → push lên GitHub Container Registry
|
| └─ Job 3 – DEPLOY (needs: build)
|   server pull image mới → chạy container mới thay container cũ
```

Phần II sẽ hướng dẫn bạn tự tay xây dựng đúng pipeline này, từng bước một.

PHẦN II — LAB THỰC HÀNH XUYÊN SUỐT

Xây dựng CI/CD hoàn chỉnh: FastAPI + pytest + Docker + GitHub Actions

Mục tiêu cuối lab: mỗi lần bạn `git push`, GitHub tự động **test** → **build Docker image** → **push image lên registry** → **(deploy)**. Toàn bộ dùng công cụ miễn phí.

Chuẩn bị (Lab 0):

- Python ≥ 3.10 (`python --version`)
 - Git + tài khoản GitHub
 - Docker Desktop (Windows/Mac) hoặc Docker Engine (Linux) — kiểm tra: `docker --version`
 - Editor: VS Code (khuyến nghị)
-

Lab 1 — Tạo webapp FastAPI

Bước 1.1 — Tạo project và môi trường ảo

```
bash
```

```
mkdir fastapi-cicd-demo && cd fastapi-cicd-demo

# Tạo môi trường ảo để cô lập thư viện của project
python -m venv venv

# Kích hoạt môi trường ảo
source venv/bin/activate          # Linux/Mac
# venv\Scripts\activate           # Windows
```

Bước 1.2 — Khai báo dependencies

Tạo file `requirements.txt`:

```
txt

fastapi==0.115.12
uvicorn[standard]==0.34.0
pytest==8.3.5
httpx==0.28.1
```

Giải thích:

- `fastapi` — framework web
- `uvicorn` — ASGI server để chạy FastAPI (`[standard]` kèm các tối ưu hiệu năng)
- `pytest` — framework viết test
- `httpx` — thư viện HTTP mà TestClient của FastAPI cần để giả lập request khi test

Cài đặt:

```
bash

pip install -r requirements.txt
```

Bước 1.3 — Viết ứng dụng

Tạo cấu trúc:

```
fastapi-cicd-demo/  
├─ app/  
|   ├── __init__.py      # file rỗng, đánh dấu app là 1 Python package  
|   └─ main.py           # code ứng dụng  
├─ tests/  
|   ├── __init__.py  
|   └─ test_main.py      # code test (Lab 2)  
└─ requirements.txt
```

File `app/main.py`:

```
python
```

```

"""
Webapp demo cho lab CI/CD.
API máy tính đơn giản: trang chủ, health check, cộng, chia.
"""

from fastapi import FastAPI, HTTPException

# Khởi tạo ứng dụng. title/version hiển thị trong trang docs tự sinh.
app = FastAPI(title="CI/CD Demo App", version="1.0.0")

@app.get("/")
def read_root():
    """Trang chủ – trả về lời chào."""
    return {"message": "Hello DevOps! App đang chạy."}

@app.get("/health")
def health_check():
    """
    Health check endpoint – QUAN TRỌNG trong DevOps:
    hệ thống giám sát / Docker / load balancer gọi endpoint này
    định kỳ để biết app còn sống hay không.
    """
    return {"status": "ok"}

@app.get("/add")
def add(a: float, b: float):
    """
    Cộng 2 số, truyền qua query string: /add?a=1&b=2
    FastAPI tự validate kiểu dữ liệu: truyền chữ sẽ bị trả lỗi 422.
    """
    return {"a": a, "b": b, "result": a + b}

@app.get("/divide")
def divide(a: float, b: float):
    """Chia 2 số. Chia cho 0 trả lỗi 400 – case này sẽ được test kỹ."""
    if b == 0:
        raise HTTPException(status_code=400, detail="Không thể chia cho 0")
    return {"a": a, "b": b, "result": a / b}

```

Bước 1.4 — Chạy thử

```
bash
```

```
uvicorn app.main:app --reload  
# app.main:app = biến `app` trong file app/main.py  
# --reload = tự khởi động lại khi code đổi (CHỈ dùng khi dev)
```

Mở trình duyệt kiểm tra:

- <http://127.0.0.1:8000> → `{"message": "Hello DevOps! App đang chạy."}`
- <http://127.0.0.1:8000/docs> → **Swagger UI** — trang tài liệu API tự sinh, test API trực tiếp tại đây
- <http://127.0.0.1:8000/add?a=5&b=3> → `{"a":5,"b":3,"result":8}`

✅ **Checkpoint Lab 1:** cả 3 URL trên hoạt động. Nhấn `Ctrl+C` để dừng server.

Lab 2 — Viết test tự động với pytest

Test tự động là **trái tim** của CI — không có test, CI chẳng có gì để "kiểm tra".

Bước 2.1 — Viết test

File `tests/test_main.py`:

```
python
```

```
"""
```

Test cho app/main.py.

TestClient giả lập HTTP request đến app NGAY TRONG bộ nhớ – không cần khởi động server thật, chạy rất nhanh.

```
"""
```

```
from fastapi.testclient import TestClient
```

```
from app.main import app
```

```
client = TestClient(app)
```

```
def test_read_root():
```

```
    """Trang chủ phải trả 200 và có message."""
```

```
    response = client.get("/")
```

```
    assert response.status_code == 200
```

```
    assert "message" in response.json()
```

```
def test_health_check():
```

```
    """Health check phải trả status ok."""
```

```
    response = client.get("/health")
```

```
    assert response.status_code == 200
```

```
    assert response.json() == {"status": "ok"}
```

```
def test_add():
```

```
    """Cộng 2 + 3 phải bằng 5."""
```

```
    response = client.get("/add", params={"a": 2, "b": 3})
```

```
    assert response.status_code == 200
```

```
    assert response.json()["result"] == 5
```

```
def test_add_invalid_input():
```

```
    """Truyền chữ thay vì số → FastAPI phải trả 422 (validation error)."""
```

```
    response = client.get("/add", params={"a": "xyz", "b": 3})
```

```
    assert response.status_code == 422
```

```
def test_divide():
```

```
    """Chia 10 / 4 = 2.5"""
```

```
    response = client.get("/divide", params={"a": 10, "b": 4})
```

```
    assert response.json()["result"] == 2.5
```

```
def test_divide_by_zero():
    """Chia cho 0 phải trả lỗi 400 – test cả trường hợp LỖI, không chỉ happy path"""
    response = client.get("/divide", params={"a": 10, "b": 0})
    assert response.status_code == 400
```

Bước 2.2 — Chạy test

```
bash

pytest -v
# pytest tự tìm các file test_*.py và hàm test_*
# -v (verbose): in chi tiết từng test
```

Kết quả mong đợi:

```
tests/test_main.py::test_read_root PASSED
tests/test_main.py::test_health_check PASSED
tests/test_main.py::test_add PASSED
tests/test_main.py::test_add_invalid_input PASSED
tests/test_main.py::test_divide PASSED
tests/test_main.py::test_divide_by_zero PASSED
===== 6 passed in 0.5s =====
```

💡 **Thử nghiệm hiểu bài:** vào `app/main.py`, đổi `(a + b)` thành `(a - b)`, chạy lại `pytest` → test fail. Đây chính xác là điều CI làm giúp bạn ở mỗi lần push. Nhớ sửa lại cho đúng!

✅ **Checkpoint Lab 2:** 6/6 test passed.

Lab 3 — Docker hóa ứng dụng

Bước 3.1 — Tạo `.dockerignore`

Tạo file `.dockerignore` ở thư mục gốc (những gì KHÔNG đưa vào image):


```
venv/  
__pycache__/  
*.pyc  
.git/  
.github/  
.pytest_cache/  
tests/  
.env  
README.md
```

Bước 3.2 — Viết Dockerfile

Tạo file `Dockerfile` (không có đuôi file) ở thư mục gốc:

```
dockerfile
```

```

# =====
# Dockerfile cho FastAPI app – mỗi dòng có giải thích
# =====

# 1) Image nền: Debian tối giản + Python 3.12 cài sẵn.
#     Bản -slim nhẹ (~120MB), đủ dùng cho đa số app Python.
FROM python:3.12-slim

# 2) Biến môi trường:
#     PYTHONDONTWRITEBYTECODE=1 : không sinh file .pyc (rác trong container)
#     PYTHONUNBUFFERED=1         : in log ra ngay, không buffer → dễ xem docker l
ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1

# 3) Thư mục làm việc trong container – mọi lệnh sau chạy tại đây
WORKDIR /code

# 4) Copy RIÊNG requirements.txt trước để tận dụng layer cache:
#     code đổi hằng ngày nhưng thư viện ít đổi → layer pip install
#     được tái sử dụng → build lại nhanh hơn rất nhiều.
COPY requirements.txt .

# 5) Cài thư viện lúc BUILD image.
#     --no-cache-dir: không giữ cache pip → image nhỏ hơn
RUN pip install --no-cache-dir -r requirements.txt

# 6) Bây giờ mới copy source code vào image
COPY ./app ./app

# 7) Khai báo container lắng nghe cổng 8000 (mang tính tài liệu)
EXPOSE 8000

# 8) Lệnh chạy khi container KHỞI ĐỘNG (dạng exec – mảng JSON,
#     để app nhận SIGTERM và shutdown êm).
#     --host 0.0.0.0 BẮT BUỘC: nếu không, app chỉ nghe localhost
#     bên trong container, từ ngoài không truy cập được.
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]

```

Bước 3.3 — Build và chạy container

```
bash
```

```
# Build image, đặt tên (tag) là fastapi-demo, phiên bản v1
# Dấu "." = build context: thư mục hiện tại
docker build -t fastapi-demo:v1 .

# Xem image vừa tạo
docker images

# Chạy container:
# -d : chạy nền (detached)
# -p 8000:8000 : map cổng 8000 máy bạn → cổng 8000 container
# --name demo : đặt tên container cho dễ thao tác
docker run -d -p 8000:8000 --name demo fastapi-demo:v1

# Kiểm tra container đang chạy
docker ps

# Gọi thử API (hoặc mở trình duyệt http://localhost:8000)
curl http://localhost:8000/health
# → {"status":"ok"}

# Xem log của app
docker logs demo

# Dọn dẹp khi xong
docker stop demo && docker rm demo
```

✅ **Checkpoint Lab 3:** `curl http://localhost:8000/health` trả `{"status":"ok"}` — app của bạn giờ chạy trong container, mang đi máy nào cũng chạy y hệt.

Lab 4 — CI với GitHub Actions: tự động test mỗi lần push

Bước 4.1 — Đưa code lên GitHub

Tạo file `.gitignore`:

```
venv/
__pycache__/
*.pyc
.pytest_cache/
.env
```

Tạo repo mới trên GitHub (ví dụ `fastapi-cicd-demo`), rồi:

bash

```
git init
git add .
git commit -m "Lab 1-3: FastAPI app + tests + Dockerfile"
git branch -M main
git remote add origin https://github.com/<username>/fastapi-cicd-demo.git
git push -u origin main
```

Bước 4.2 — Viết workflow CI

Tạo thư mục và file: `.github/workflows/ci.yml` (đúng đường dẫn này, GitHub mới nhận diện):

yaml

```

# =====
# Workflow CI: tự động chạy test mỗi lần push / tạo PR
# =====
name: CI - Test

# ---- Trigger: khi nào workflow chạy ----
on:
  push:
    branches: [ main ]      # push lên main
  pull_request:
    branches: [ main ]      # hoặc mở PR nhắm vào main

jobs:
  test:
    runs-on: ubuntu-latest   # máy ảo Ubuntu do GitHub cấp, miễn phí

    steps:
      # Bước 1: tải code của repo về runner.
      # Runner là máy trắng – không checkout thì không có code để test!
      - name: Checkout code
        uses: actions/checkout@v4

      # Bước 2: cài Python bằng action chính thức
      - name: Set up Python 3.12
        uses: actions/setup-python@v5
        with:
          python-version: "3.12"
          cache: "pip"        # cache thư mục pip → lần chạy sau nhanh hơn

      # Bước 3: cài thư viện của project
      - name: Install dependencies
        run: |
          python -m pip install --upgrade pip
          pip install -r requirements.txt

      # Bước 4: chạy test – nếu bất kỳ test nào fail,
      # step này fail → job fail → commit bị đánh dấu ❌
      - name: Run tests with pytest
        run: pytest -v

```

Bước 4.3 — Push và quan sát

```
bash
```

```
git add .github/workflows/ci.yml
git commit -m "Lab 4: add CI workflow"
git push
```

Mở repo trên GitHub → tab **Actions** → thấy workflow "CI - Test" đang chạy → bấm vào xem log từng step. Sau ~1 phút sẽ có dấu  xanh.

💡 **Thử nghiệm quan trọng nhất lab này:** cố tình làm hỏng code (đổi `a + b` thành `a - b` trong `app/main.py`), commit và push → vào tab Actions xem workflow **fail** , đọc log để thấy chính xác test nào fail và vì sao. Đây là giá trị thật của CI: **máy phát hiện lỗi thay bạn, trước khi lỗi đến production**. Sửa lại code và push để workflow xanh trở lại.

 **Checkpoint Lab 4:** tab Actions có ít nhất 1 lần chạy  và bạn đã chứng kiến 1 lần chạy .

Lab 5 — CD phần 1: Build & Push Docker image tự động

Khi test pass, pipeline sẽ tự build image và đẩy lên **GitHub Container Registry (GHCR)** — registry miễn phí gắn liền repo, không cần đăng ký gì thêm.

Bước 5.1 — Nâng cấp workflow thành pipeline đầy đủ

Sửa `.github/workflows/ci.yml` thành:

yaml

```

# =====
# Pipeline CI/CD hoàn chỉnh: Test → Build & Push image → Deploy
# =====
name: CI/CD Pipeline

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

# Biên dùng chung cho toàn workflow
env:
  REGISTRY: ghcr.io # GitHub Container Registry
  IMAGE_NAME: ${ github.repository } # = <username>/fastapi-cicd-demo

jobs:
  # ===== JOB 1: TEST (CI) =====
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
          cache: "pip"

      - name: Install dependencies
        run: |
          python -m pip install --upgrade pip
          pip install -r requirements.txt

      - name: Run tests
        run: pytest -v

  # ===== JOB 2: BUILD & PUSH IMAGE (CD) =====
  build-and-push:
    runs-on: ubuntu-latest
    needs: test # CHỈ chạy khi job test đã PASS
    # Chỉ build khi push lên main (không build cho PR):
    if: github.event_name == 'push' && github.ref == 'refs/heads/main'

```

```

# Cấp quyền cho GITHUB_TOKEN (token tự sinh cho mỗi lần chạy):
permissions:
  contents: read      # đọc code
  packages: write     # ghi (push) image lên GHCR

steps:
  - uses: actions/checkout@v4

  # Đăng nhập GHCR — không cần tạo secret thủ công,
  # GITHUB_TOKEN được GitHub cấp sẵn cho mỗi workflow run
  - name: Log in to GitHub Container Registry
    uses: docker/login-action@v3
    with:
      registry: ${ env.REGISTRY }
      username: ${ github.actor }      # user đang chạy workflow
      password: ${ secrets.GITHUB_TOKEN } # token tự động, an toàn

  # Sinh tag/label chuẩn cho image (latest + mã commit)
  - name: Extract metadata (tags, labels)
    id: meta                                # đặt id để step sau tham chi
    uses: docker/metadata-action@v5
    with:
      images: ${ env.REGISTRY }/${ env.IMAGE_NAME }
      tags: |
        type=raw,value=latest
        type=sha                                # tag theo commit SHA — truy

  # Build image từ Dockerfile và push lên GHCR trong 1 bước
  - name: Build and push Docker image
    uses: docker/build-push-action@v6
    with:
      context: .                                # build context = gốc repo
      push: true                                # push sau khi build xong
      tags: ${ steps.meta.outputs.tags }        # lấy output từ step "meta"
      labels: ${ steps.meta.outputs.labels }

```

Giải thích những điểm mới:

- `needs: test` — tạo pipeline **tuần tự**: test fail thì không bao giờ build. Đây là "cửa chặn chất lượng" (quality gate).
- `if:` — job build chỉ chạy với push lên `main`; PR chỉ chạy test.
- `permissions:` — nguyên tắc **least privilege**: chỉ cấp đúng quyền cần.

- `{{ secrets.GITHUB_TOKEN }}` — secret **tự động có sẵn**, không phải tạo tay; dùng để xác thực với GHCR.
- `id: meta` + `{{ steps.meta.outputs.tags }}` — cách một step **truyền dữ liệu** cho step sau.

Bước 5.2 — Push và kiểm tra

```
bash
git add . && git commit -m "Lab 5: full CI/CD pipeline with GHCR" && git push
```

Vào tab **Actions**: thấy 2 job chạy **tuần tự** (test → build-and-push). Xong, vào trang chính của repo → mục **Packages** (cột phải) → thấy image `fastapi-cicd-demo` với tag `latest` và `sha-xxxxxxx`.

Bất kỳ máy nào có Docker giờ có thể chạy app của bạn:

```
bash
docker pull ghcr.io/<username>/fastapi-cicd-demo:latest
docker run -d -p 8000:8000 ghcr.io/<username>/fastapi-cicd-demo:latest
```

(Nếu pull bị 403: repo/package đang private → vào Package settings đổi visibility thành Public, hoặc `docker login ghcr.io` trước.)

✅ **Checkpoint Lab 5:** image xuất hiện trong Packages; pull và chạy được trên máy local.

Lab 6 — CD phần 2: Deploy tự động

Có image trên registry rồi, "deploy" đơn giản là: **server pull image mới và chạy lại container**. Chọn một trong hai hướng:

Hướng A — Deploy lên server riêng qua SSH (VPS: EC2, DigitalOcean, Azure VM...)

Chuẩn bị trên server (một lần): cài Docker; tạo cặp SSH key; thêm public key vào `~/.ssh/authorized_keys` của server.

Tạo secrets cho repo — vào repo → *Settings* → *Secrets and variables* → *Actions* → *New repository secret*, tạo 3 secret:

Secret	Giá trị
<code>SERVER_HOST</code>	IP hoặc domain của server
<code>SERVER_USER</code>	user SSH (vd: <code>ubuntu</code>)
<code>SSH_PRIVATE_KEY</code>	nội dung file private key

⚠ Secret được GitHub mã hóa và **tự động che (mask) trong log** — đây là cách duy nhất đúng để đưa thông tin nhạy cảm vào workflow. Không bao giờ hardcode password/token vào file YAML.

Thêm job deploy vào cuối file workflow:

```
yaml

# ===== JOB 3: DEPLOY (CD) =====
deploy:
  runs-on: ubuntu-latest
  needs: build-and-push          # chỉ deploy khi đã có image mới
  if: github.ref == 'refs/heads/main'

  steps:
    - name: Deploy to server via SSH
      uses: appleboy/ssh-action@v1    # action SSH vào server và chạy lệnh
      with:
        host: ${ secrets.SERVER_HOST }
        username: ${ secrets.SERVER_USER }
        key: ${ secrets.SSH_PRIVATE_KEY }
        script: |
          # Kéo image phiên bản mới nhất về server
          docker pull ghcr.io/${ github.repository }:latest
          # Dừng & xóa container cũ (|| true: bỏ qua lỗi nếu chưa tồn tại)
          docker stop myapp || true
          docker rm myapp || true
          # Chạy container mới từ image mới
          docker run -d --name myapp --restart unless-stopped \
            -p 80:8000 \
            ghcr.io/${ github.repository }:latest
          # Dọn image cũ không dùng để tiết kiệm đĩa
          docker image prune -f
```

Push xong, mở `http://<SERVER_HOST>/health` → `{"status": "ok"}`. Từ giờ, **mỗi lần push lên main = một lần release tự động**: test → build → deploy, không cần đụng tay vào server.

Hướng B — Chưa có server? Dùng nền tảng PaaS miễn phí

Nếu chưa có VPS, dùng Render hoặc Railway: tạo Web Service mới → kết nối repo GitHub → chọn "Docker" → nền tảng tự phát hiện Dockerfile, tự build và deploy **mỗi lần bạn push** (họ tự làm phần CD). Workflow của bạn vẫn giữ vai trò CI (chặn code lỗi bằng test trước khi merge).

✅ **Checkpoint Lab 6:** thay đổi message ở endpoint `/` (vd: "Hello DevOps v2!"), push, đợi pipeline xanh, mở URL production → thấy message mới. **Chúc mừng — bạn vừa hoàn thành một pipeline CI/CD hoàn chỉnh đúng chuẩn!**

Tổng kết — Bạn đã học được gì

Lab	Kỹ năng	Khái niệm DevOps
1	Xây webapp FastAPI	Ứng dụng có health check
2	Viết test pytest	Nền tảng của CI
3	Dockerfile, build/run container	Đóng gói, Infrastructure as Code
4	Workflow, event, job, step, action	Continuous Integration
5	Pipeline nhiều job, needs, GHCR, secrets	Continuous Delivery, artifact
6	Deploy tự động qua SSH / PaaS	Continuous Deployment

Cấu trúc project cuối cùng:

```
fastapi-cicd-demo/
├── .github/workflows/ci.yml    # pipeline CI/CD
├── app/
│   ├── __init__.py
│   └── main.py                # FastAPI app
├── tests/
│   ├── __init__.py
│   └── test_main.py          # pytest tests
├── .dockerignore
├── .gitignore
├── Dockerfile
└── requirements.txt
```

Hướng phát triển tiếp theo (tự luyện):

1. Thêm **lint** vào job test: `pip install ruff` + step `run: ruff check .`
2. Thêm **code coverage**: `pytest --cov=app`
3. **Docker Compose**: chạy app + PostgreSQL cùng lúc
4. **Environments & approval**: yêu cầu người duyệt trước khi deploy production (Continuous Delivery đúng nghĩa)
5. Tìm hiểu **Kubernetes** khi cần chạy nhiều container ở quy mô lớn

TÀI LIỆU THAM KHẢO (truy cập được)

GitHub Actions

- Tài liệu chính: <https://docs.github.com/en/actions>
- Cú pháp workflow (giải thích mọi từ khóa):
<https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-syntax>
- Các event kích hoạt workflow:
<https://docs.github.com/en/actions/reference/workflows-and-actions/events-that-trigger-workflows>
- Secrets: <https://docs.github.com/en/actions/security-for-github-actions/security-guides/using-secrets-in-github-actions>
- CI cho Python: <https://docs.github.com/en/actions/automating-builds-and-tests/building-and-testing-python>
- Publish Docker image: <https://docs.github.com/en/actions/publishing-packages/publishing-docker-images>

Docker

- Get started: <https://docs.docker.com/get-started/>
- Dockerfile reference: <https://docs.docker.com/reference/dockerfile/>
- Hướng dẫn Docker cho Python: <https://docs.docker.com/language/python/>
- GitHub Container Registry: <https://docs.github.com/en/packages/working-with-a-github-packages-registry/working-with-the-container-registry>

FastAPI & pytest

- FastAPI tutorial: <https://fastapi.tiangolo.com/tutorial/>
- FastAPI + Docker (chính chủ, giải thích Dockerfile rất kỹ): <https://fastapi.tiangolo.com/deployment/docker/>
- FastAPI testing: <https://fastapi.tiangolo.com/tutorial/testing/>
- pytest: <https://docs.pytest.org/>

DevOps & CI/CD tổng quan

- GitHub — About CI: <https://docs.github.com/en/actions/automating-builds-and-tests/about-continuous-integration>
- Atlassian DevOps guide: <https://www.atlassian.com/devops>
- Red Hat — What is CI/CD: <https://www.redhat.com/en/topics/devops/what-is-ci-cd>